



Microsoft Foundry & AI Studio



MICHAEL PLETTNER

Managing Partner



Michael Plettner



Microsoft Teams
User Group Germany



<https://bio.link/plemich>



Decode AI
neat.



Wann reicht der managed Weg nicht?

• Copilot Studio

- Citizen Developer, kein Code
- Standard-RAG auf M365-Daten
- Out-of-the-box-Konnektoren
- Schneller Rollout

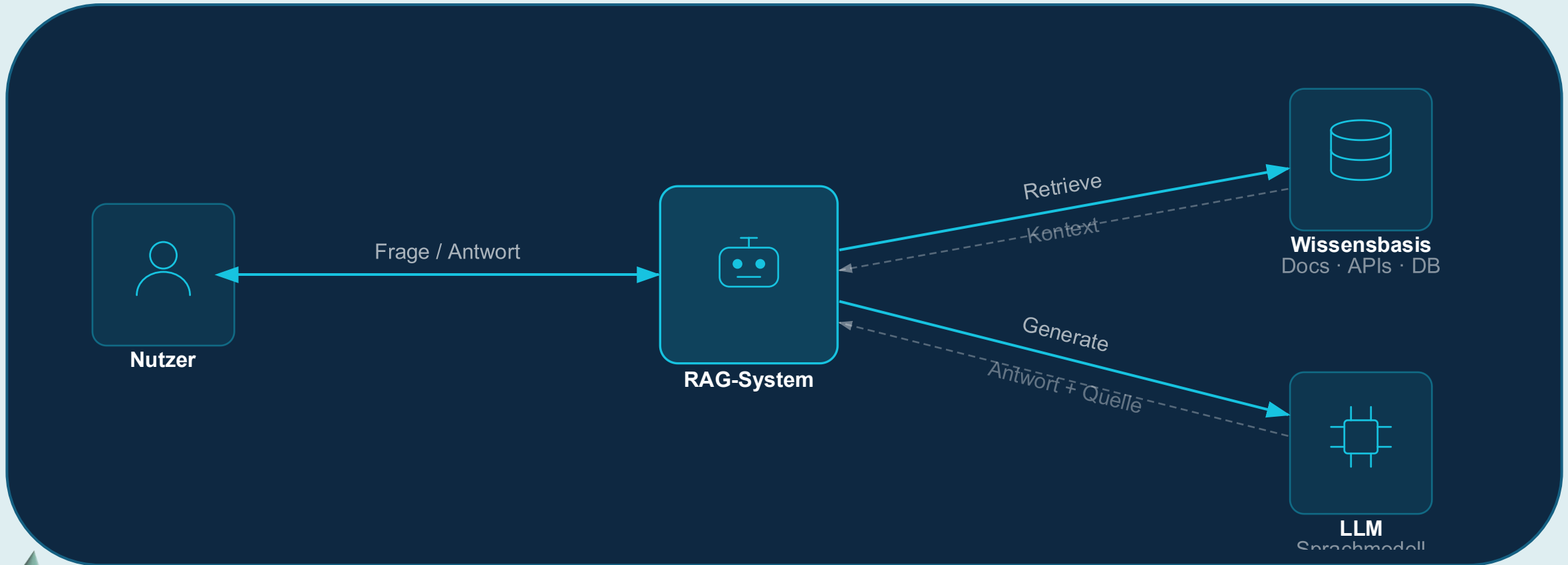
• Microsoft Foundry

- Custom RAG, eigene Infra
- Multi-Modell, volles Tracing
- Komplexe Tool-Orchestrierung
- Kosten- & Infra-Kontrolle

Teil 1 · Die Begriffe

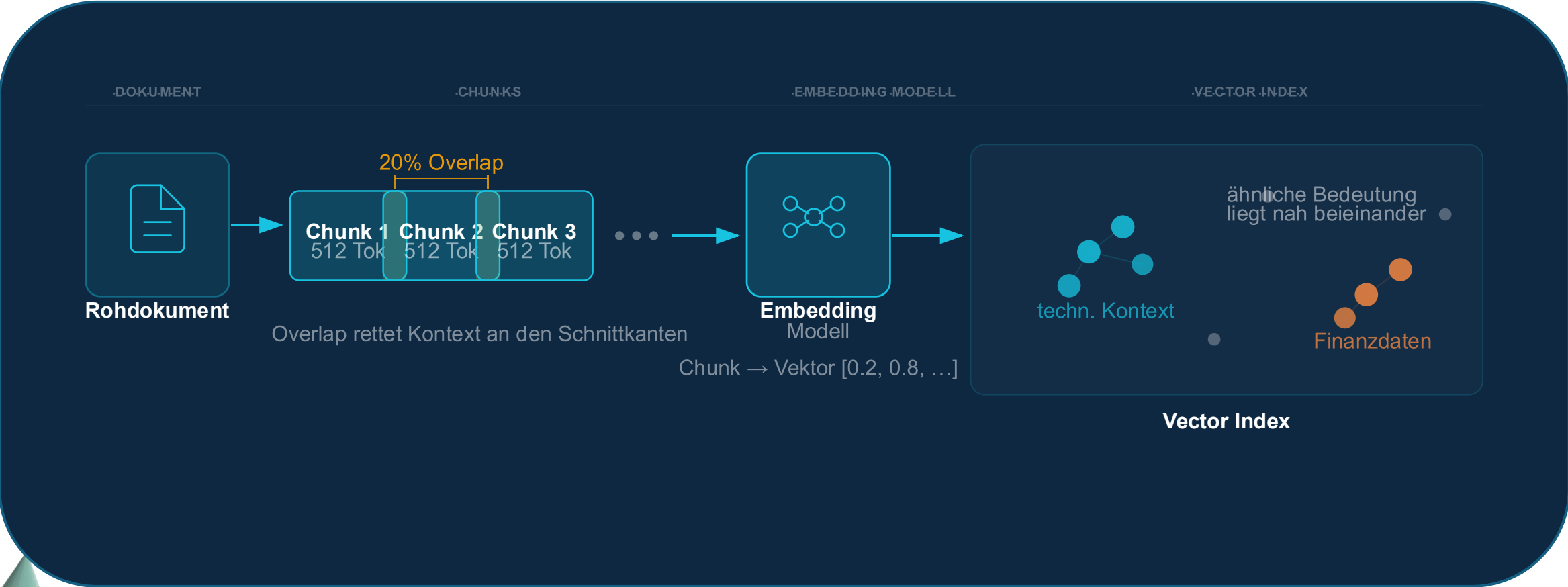
RAG · Chunking & Embeddings · Agent & Tool-Calls · Tracing

RAG – erst nachschlagen, dann antworten



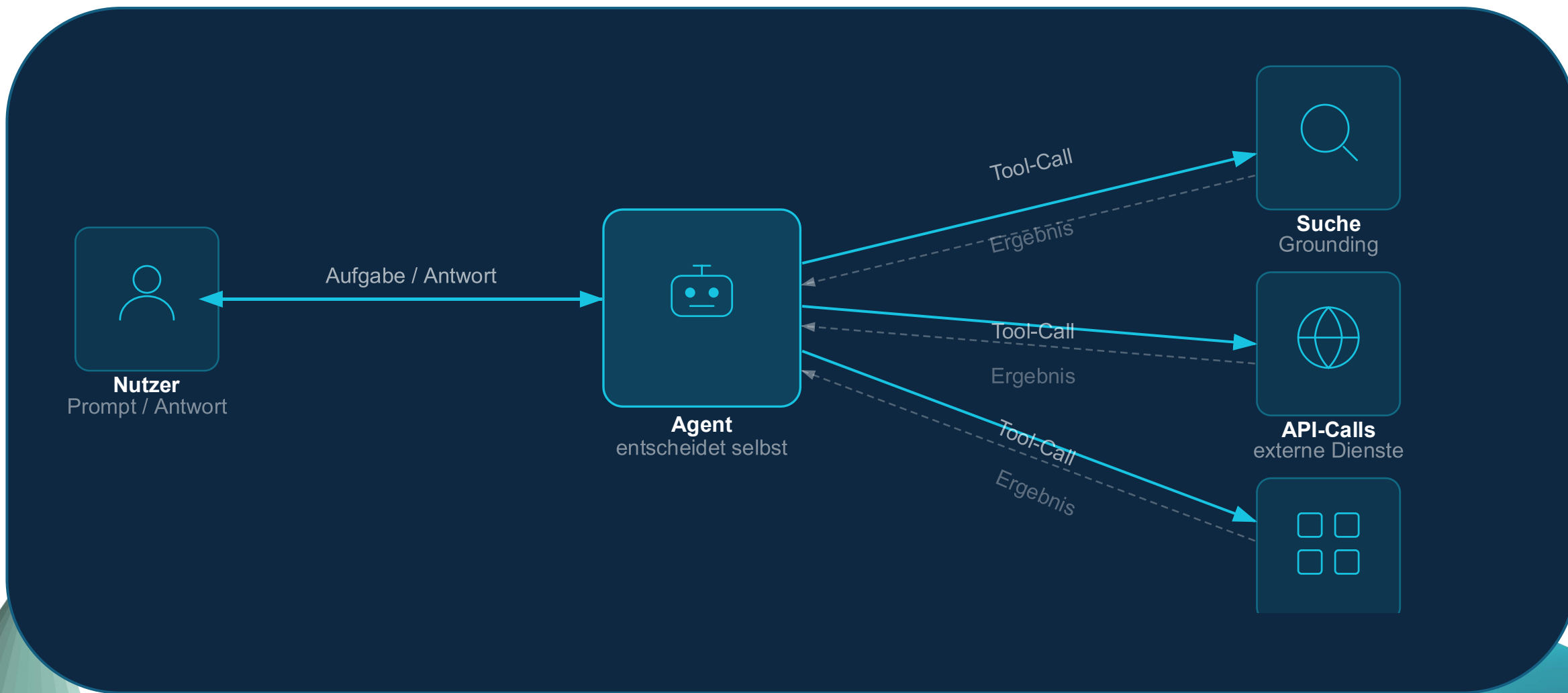
Entscheidend ist nicht der Prompt – sondern was gefunden wird

Chunking & Embeddings



Der Hebel für Retrieval-Qualität sitzt im **Chunking** – nicht im Prompt

Agent & Tool-Calls



Orchestrierung im Code – nicht im Klickmenü

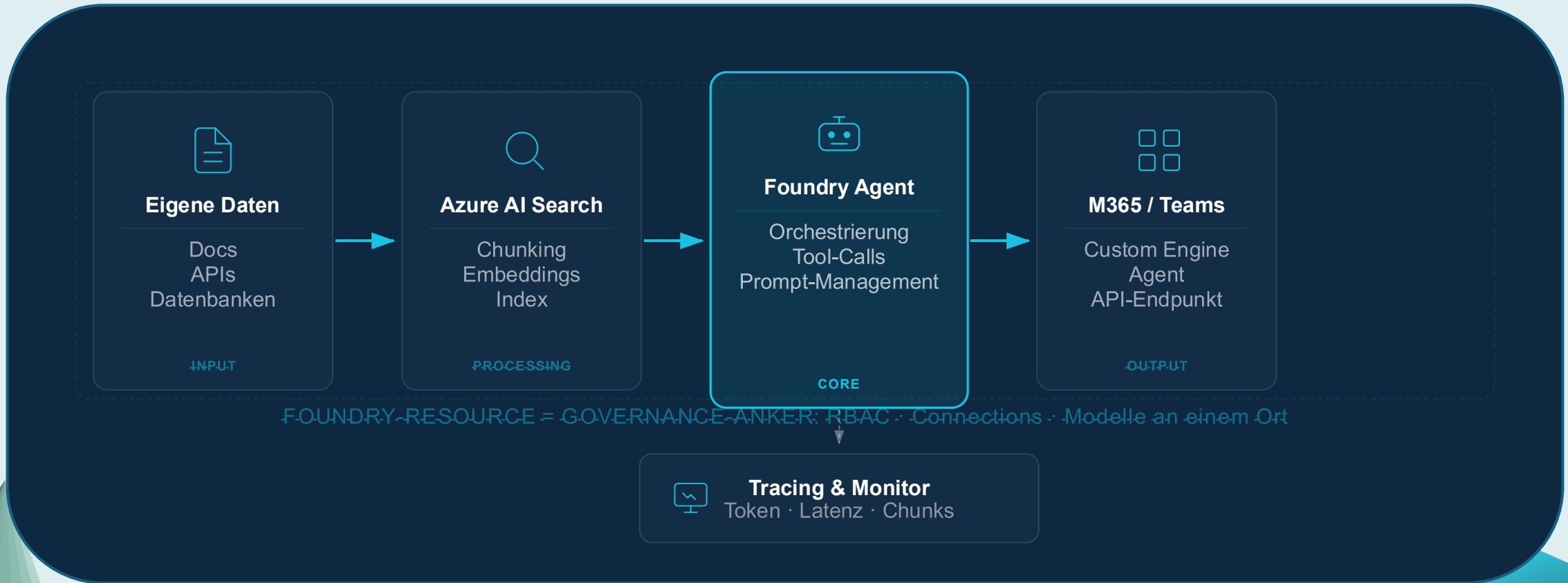
Tracing & Observability



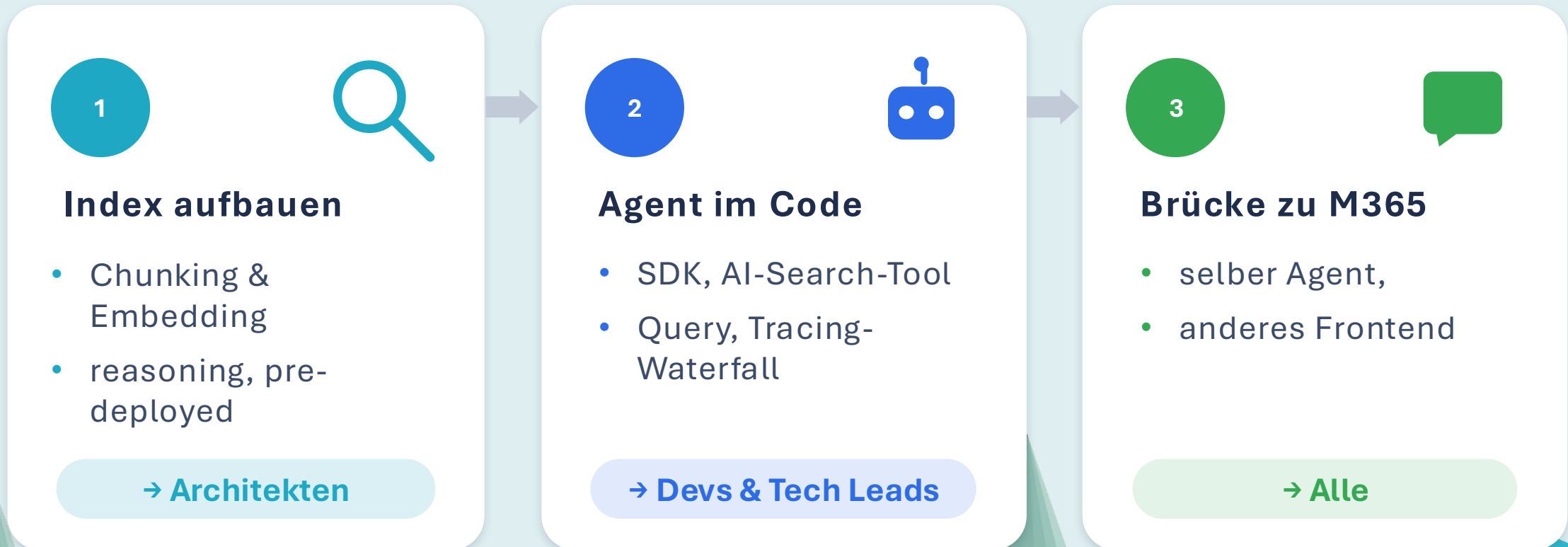
Reinschauen statt raten – Debugging und Kostenkontrolle in einem

Teil 2 · Die Praxis

Microsoft Foundry auf einer Folie



Ein Wissens-Agent über interne Doku



Eine Frage läuft durch alle drei Stufen — gleiche Antwort, anderes Frontend.

Die drei zentralen Fragen



Sicherheit

- RBAC auf Resource & Project
- Private Endpoints
- Managed Identity — keine Secrets



Kosten

- Token-Tracking pro Deployment
- Quotas & Monitoring
- Modellwahl = Preiseffekt



Qualität

- Foundry Evaluation
- Groundedness · Relevance
- Agent Optimizer

Takeaways

1

Volle Kontrolle = Feature, nicht Bug

- Foundry gibt Kontrolle zurück — auf Kosten von Komplexität.

2

RAG-Qualität entscheidet beim Index

- Chunking und Embedding sind die Hebel — nicht der Prompt.

3

Foundry + Copilot Studio = ein Stack

- Foundry als Backend, Copilot Studio als Frontend.

Fragen?



Vielen Dank!

Ich freue mich auf Feedback!

Scan
mich!

